Sequential Gaussian Processes for Online Learning of Nonstationary Functions

Michael Minyi Zhang, Bianca Dumitrascu, Sinead A. Williamson, and Barbara E. Engelhardt

***Abstract*—Many machine learning problems can be framed in the context of estimating functions, and often these are time-dependent functions that are estimated in real-time as observations arrive. Gaussian processes (GPs) are an attractive choice for modeling real-valued nonlinear functions due to their flexibility and uncertainty quantification. However, the typical GP regression model suffers from several drawbacks: 1) Conventional GP inference scales $\mathcal{O}(N^3)$ with respect to the number of observations; 2) Updating a GP model sequentially is not trivial; and 3) Covariance kernels typically enforce stationarity constraints on the function, while GPs with non-stationary covariance kernels are often intractable to use in practice. To overcome these issues, we propose a sequential Monte Carlo algorithm to fit infinite mixtures of GPs that capture non-stationary behavior while allowing for online, distributed inference. Our approach empirically improves performance over state-of-the-art methods for online GP estimation in the presence of non-stationarity in time-series data. To demonstrate the utility of our proposed online Gaussian process mixture-of-experts approach in applied settings, we show that we can sucessfully implement an optimization algorithm using online Gaussian process bandits.**

***Index Terms*—Gaussian processes, sequential Monte Carlo, online learning.**

I. Introduction

DATA are often observed as streaming observations that arrive sequentially across time. Examples of streaming data include hospital patients' vital signs, telemetry data, online purchases, and stock prices. To model streaming data, it is more efficient to update model parameters as new observations arrive than to refit the model from scratch with the new observations appended onto existing data. A typical prior distribution on the space of functions for time series analysis is the Gaussian process [1]. Gaussian processes (GPs) are a convenient distribution on real-valued functions because, when evaluated at a fixed set of inputs, they have a multivariate normal distribution and hence allow closed-form posterior inference and prediction when used for regression.

Parameter estimation for GPs remains challenging because inference involves calculating the Gaussian likelihood. This means that the computational complexity of inference is dominated by the $\mathcal{O}(N^3)$ operation of inverting an $N \times N$ matrix, where N is the number of observations. This complexity makes standard GP inference techniques unsuitable for scenarios with large numbers of observations or where fast inference is essential. Further, Bayesian approaches, such as Markov chain Monte Carlo (MCMC) or variational inference (VI), do not trivially allow online updating of model parameter estimates, although sequential Monte Carlo methods (SMC) may be adapted for this purpose.

From a statistical perspective, a typical GP regression model infers only stationary functions, meaning that properties of the function are constant across all input values. While there are covariance kernels that explicitly capture non-stationary effects in GP regression, they pose greater computational challenges than a stationary kernel as they often require calculating intractable integrals. Mixture-of-experts GP models have been used to model non-stationary functions by fitting independent GPs to different segments of the input space. In particular, the importance sampled mixture-of-experts (IS-MOE) approach fits mixtures of GP experts in a distributed manner using importance sampling [2]; however, IS-MOE is not an online algorithm. Conversely, sparse online GPs are a state-of-the-art method for online GP estimation, but the estimated functions are constrained to be stationary.

Given the prevalence and complexity of streaming data, this gap necessitates a new approach for online learning of non-stationary functions. Moreover, in most mixture-of-experts approaches, we assume the number of mixtures are fixed and known a priori, which is generally inappropriate. Instead, we can assume that there is an infinite mixture of GPs, by assuming the mixing distribution is Dirichlet-process distributed. We introduce a sequential algorithm to fit infinite mixtures of GPs. SMC samplers can be adapted to allow real-time updates to the model parameters, and are trivially parallelizable. We show a connection with online GPs to multi-armed bandits for optimization and demonstrate that our method can obtain

Manuscript received 1 July 2022; revised 28 December 2022 and 15 March 2023; accepted 5 April 2023. Date of publication 17 April 2023; date of current version 5 May 2023. The associate editor coordinating the review of this manuscript and approving it for publication was Prof. George Atia. The work of Michael Zhang was supported in part by the HKU-URC Seed Fund for Basic Research for New Staff and in part by HKU Libraries Open Access Author Fund sponsored by the HKU Libraries. *(Corresponding author: Michael Minyi Zhang.)*

Michael Minyi Zhang is with the Department of Statistics and Actuarial Science, University of Hong Kong, Hong Kong, SAR, China (e-mail: mzhang18@hku.hk).

Bianca Dumitrascu is with the Department of Statistics and Herbert and Florence Institute for Cancer Dynamics, Columbia University, New York, NY 10027 USA (e-mail: bmd2151@columbia.edu).

Sinead A. Williamson is with the Department of Statistics and Data Science, The University of Texas at Austin, Austin, TX 78712 USA (e-mail: sinead.williamson@mccombs.utexas.edu).

Barbara E. Engelhardt is with the Department of Biomedical Data Science, Stanford University, Stanford, CA 94305 USA (e-mail: bengelhardt@stanford.edu).

Digital Object Identifier 10.1109/TSP.2023.3267992

superior performance compared to other GP-bandit optimization techniques.

This paper proceeds as follows: In Section II, we discuss the background for Gaussian processes, methods for fast inference, and importance sampling. We introduce our online GP inference algorithm in Section III. We show empirical benefits of our framework on prediction tasks in both simulated data and in hospital patient data where online, scalable inference is essential in Section IV. In Section V, we conclude with a discussion of future work.

II. Problem Statement and Background

A. Gaussian Processes

Consider the problem of estimating an unknown function $f : \mathbb{R}^D \to \mathbb{R}$ from streaming data. In detail, we have D-dimensional input data $\mathbf{x}_i \in \mathbb{R}^D$, and output data, $y_i \in \mathbb{R}$, which arrive at times $i = 1, 2, \ldots, N$. We assume a functional relationship, $y_i = f(\mathbf{x}_i) + \epsilon_i$, where $\epsilon_i \sim \mathcal{N}(0, \sigma^2)$ where the function f is assumed to be distributed from a Gaussian process. Gaussian processes (GPs) are a popular approach to modeling distributions over arbitrary real-valued functions $f(\cdot)$, with applications in regression, classification, and optimization, among others. Characterized by a mean function $\mu(\cdot)$, and a covariance function $\Sigma(\cdot, \cdot)$, we can sample instantiations from a GP at a fixed set of locations $\mathbf{X} = [\mathbf{x}_1, \ldots, \mathbf{x}_N]^T$, according to an N-dimensional multivariate normal:

$$f(\mathbf{X})|\mathbf{X} \sim \mathcal{N}_N(\mu(\mathbf{X}), \Sigma_{\mathbf{X}\mathbf{X}'}),$$

where $\mu(\mathbf{X})$ and $\Sigma_{\mathbf{X}\mathbf{X}'}$ are the mean and covariance functions evaluated at the input data $\mathbf{X}$. Typically, the mean function is set to be zero in Gaussian process regression. In the presence of noisy observations, $\mathbf{y} = [y_1, \ldots, y_N]^T$, whose mean value is a GP-distributed function, we observe the data generating process:

$$\mathbf{y}|\mathbf{X}, f \sim \mathcal{N}_N\left(f(\mathbf{X}), \sigma^2\mathbf{I}\right), \quad f|\mathbf{X} \sim \mathcal{N}\left(0, \Sigma_{\mathbf{X}\mathbf{X}'}\right). \tag{1}$$

Due to conjugacy between the likelihood and prior, we can marginalize out f analytically such that:

$$\mathbf{y}|\mathbf{X} \sim \mathcal{N}_N\left(0, \Sigma_{\mathbf{X}\mathbf{X}'} + \sigma^2\mathbf{I}\right). \tag{2}$$

We typically fit a GP regression model by optimizing the marginal likelihood with respect to the model hyperparameters. Given some previously unseen inputs $\mathbf{X}^*$, the posterior predictive distribution of the outputs $\mathbf{y}^*$ is

$$\begin{aligned}
\mathbf{m}_{\mathbf{y}^*} &= \Sigma_{\mathbf{X}^*\mathbf{X}}\Sigma^{-1}_{\mathbf{X}\mathbf{X}'}\mathbf{y} \\
\mathbf{S}_{\mathbf{y}^*} &= \Sigma_{\mathbf{X}^*\mathbf{X}^{*\prime}} - \Sigma_{\mathbf{X}^*\mathbf{X}}\Sigma^{-1}_{\mathbf{X}\mathbf{X}'}\Sigma_{\mathbf{X}\mathbf{X}^*} + \sigma^2\mathbf{I} \\
\mathbf{y}^*|\mathbf{y}, \mathbf{X}, \mathbf{X}^* &\sim \mathcal{N}\left(\mathbf{m}_{\mathbf{y}^*}, \mathbf{S}_{\mathbf{y}^*}\right).
\end{aligned} \tag{3}$$

B. Fast Gaussian Process Inference

Fitting GP-based models is dominated by $\mathcal{O}(N^3)$ covariance matrix inversion operations, meaning that these models are challenging to fit to large sample size data. To allow GP models to be computationally tractable for large data, numerous approaches have been developed for scalable GP inference. These approaches largely fall into two groups: sparse methods and local methods. Sparse methods approximate the GP posterior distribution with an $M \ll N$ number of inducing points that act as pseudo-inputs to the GP function, thereby reducing the computational complexity to $\mathcal{O}(NM^2)$ [3], [4]. We can further speed up these sparse methods using stochastic variational inference (SVI) that reduces the complexity by fitting the model with subsets of the M inducing points to form stochastic gradients at each iteration [5].

Local methods, on the other hand, exploit structure among samples to represent the covariance matrix using a low-rank approximation [6], [7]. To do this, they partition the data into K sets, and approximate the covariance matrix by setting the inter-partition covariances to zero. They invert this approximate covariance matrix by inverting K dense matrices of size N/K, resulting in a complexity of $\mathcal{O}(N^3/K^2)$. An additional benefit of local approaches is that we can estimate GP hyperparameters within each block; when samples are partitioned based on input location, this implicitly captures non-stationary functions. A product of experts model implies the strong assumption that, across experts, the observations are independent. This independence assumption ignores the correlation between experts and can lead to poor posterior predictive uncertainty quantification.

Instead, a more flexible approach to local methods in the GP domain is to assume that there is a mixture of GP-distributed experts. Mixture-of-experts GP models can model non-stationary functions [8], [9], [10], [11], but integration over partitions makes inference computationally intractable for most realistic settings. In contrast, variational methods for mixtures of GPs are effective to speed up computation [12], [13], [14]. Alternate methods using sum-product networks and importance sampling have also proven to be fast and effective for Bayesian inference of GP mixtures [2], [15].

C. Online Gaussian Process Inference

The online product-of-experts (POE) variant of GP regression assumes the data are strictly partitioned–leading to a block diagonal structure in the covariance matrix [16]. Contrast this to the mixture-of-experts approach, where the partition is integrated out, leading to a more expressive covariance structure, but is not inherently amenable to online updating.

Although local methods are simple to update in real-time, as we can send new data to clusters and only update clusters receiving new data, sparse and mixture-of-experts GP inference methods discussed above do not trivially allow model updates as new data arrive. Previous work in online GP inference required non-trivial adaptations to allow real-time GP inference. The sparse online Gaussian process is an inducing-point online method by approximating the posterior using variational inference. However, this approach assumes the kernel hyperparameters are fixed and known a priori [17]. The online sparse variational Gaussian process (OSVGP) is another inducing point variational method for online GP regression that updates the hyperparameters as new data arrive [18].

Many scalable inference algorithms take advantage of training the model only using a small subset of the data in a

"minibatching" approach. Examples of minibatches being used for scalable inference include stochastic gradient descent [19], stochastic variational inference (SVI) [20] and stochastic gradient MCMC [21]. However, in the sparse online settings we cannot use minibatching in a stochastic approximation setting. This is because SVI requires minibatches uniformly sampled *i.i.d.* from the entire dataset. But in the streaming case, samples from the new data may not be from the same distribution as the previous observations [18]. Moreover, minibatching in an SVI-type setting requires constant updating of the entire model, unlike in local methods that can take minibatches of the incoming data and only update a small subset of experts.

One notable variant of exact inducing point online method for streaming GP regression, called the Woodbury inversion with structured kernel interpretation (WISKI) [22], replaces the covariance kernel with a structured, sparse matrix approximation for GP regression [23]. Using this "structured kernel interpolation" method in conjunction with the Woodbury rank-one updates to the kernel inverse leads to an online GP method with constant computational complexity with respect to the number of observations. Empirically, the online variational method of OSVGP tends to suffer from numerical stability issues and is vulnerable to underfitting the model in the streaming setting [22]. Alternatively, we may interpret sparse online GP under a different formulation. Instead of the typical approaches in the aforementioned papers, the authors reframe the sparse model as a state space model and use the Kalman filtering update equations to train the model sequentially [24].

D. Modeling Non-Stationary Functions With Gaussian Processes

Modeling non-stationary functions using GPs is often computationally challenging because non-stationary kernels generally include more parameters than stationary kernels and are more computationally intensive to fit. Many non-stationary kernels model the hyperparameters as a GP-distributed function of the input space [25], [26], [27], [28]. In this setting, if we have a GP-distributed kernel hyperparameter for each observation then the computational complexity of posterior sampling the hyperparameters scales $\mathcal{O}(N^4)$ as we have to update N.

The local Gaussian process product-of-experts approach to fitting online GPs that can account for non-stationary behavior, but inference depends on a single partitioning of the input data using K-means [16]. Hard partitioning may create undesirable edge effects due to its implicit assumptions that partitions have zero correlation. The Bayesian treed GP addresses the problem of hard partitioning by integrating over the space of decision tree partitions via reversible jump MCMC [8]. While this method flexibly captures non-stationary functions, reversible jump is slow to marginalize over the space of treed GP partitions and is not parallelizable due the Markov dependencies in MCMC.

The Gaussian process change point model captures non-stationary behavior in GPs by jointly modeling a GP regression model and a change point generating process [29]. The prior distribution on the change point locations is assumed to follow a survival function that is known a priori, which represents the run time of a functional regime. This model estimates the change point runtime in an online setting, based on a Bayesian online change point detection model [30].

E. Dirichlet Process Mixture Modeling

For a finite mixture model, we assume that there are K mixtures where the observations $\mathbf{y}$ are parameterized by $\theta_k \sim \mathcal{H}$ for some distribution $\mathcal{H}$ defined on a parameter space Θ with mixture weights $\boldsymbol{\pi} = [\pi_1, \ldots, \pi_K]$, where $0 < \pi_k < 1$ and $\sum_{k=1}^{K} \pi_k = 1$:

$$P(\mathbf{y}|\boldsymbol{\theta}, \boldsymbol{\pi}) = \prod_{i=1}^{N} \sum_{k=1}^{K} \pi_k P(y_i|\theta_k). \tag{4}$$

For the purpose of computational tractability, we can augment the mixture model with a latent indicator $z_i \in \{1, \ldots, K\}$, which tells us the latent mixture membership for observation y_i, so that our mixture model is now parameterized as:

$$P(\mathbf{y}|\boldsymbol{\theta}, \mathbf{z}, \boldsymbol{\pi}, \alpha) = \prod_{i=1}^{N} P(y_i|\theta_{z_i}), \quad P(z_i|\boldsymbol{\pi}) \sim \text{Categorical}(\boldsymbol{\pi})$$

$$P(\boldsymbol{\pi}|\alpha) \sim \text{Dirichlet}\left(\frac{\alpha}{K}, \ldots, \frac{\alpha}{K}\right), P(\theta_k) \sim \mathcal{H}. \tag{5}$$

In our proposed approach, we assume that now there is an a priori infinite number of mixtures and that the mixture distribution is generated from a Dirichlet process. The Dirichlet process (DP) is a distribution over probability measures $\mathcal{G} \sim \text{DP}(\alpha, \mathcal{H})$, defined on some parameter space, Θ, with a concentration parameter $\alpha \in \mathbb{R}^+$. We define a Dirichlet process by its finite dimensional marginals. For a finite partition of Θ, $(A_1, \ldots, A_K)$, if the probability masses $\mathcal{G}(A_1), \ldots, \mathcal{G}(A_K)$ are Dirichlet$(\alpha\mathcal{H}(A_1), \ldots, \alpha\mathcal{H}(A_K))$-distributed then $\mathcal{G} \sim \text{DP}(\alpha, \mathcal{H})$ [31]. We obtain the Dirichlet process mixture model (DPMM) by taking the limit in (5) as $K \to \infty$.

Furthermore, we can integrate out the mixing proportion $\boldsymbol{\pi}$ and sample the latent indicators $\mathbf{z}$ from the predictive distribution of the Dirichlet process–the Chinese restaurant process [32], [33]. In the Chinese restaurant process (CRP), the i-th latent indicator probability is now:

$$P(z_i = k|z_1, \ldots, z_{i-1}) = \begin{cases} \frac{N'_k}{i-1+\alpha} & k \in (z_1, \ldots, z_{i-1}), \\ \frac{\alpha}{i-1+\alpha} & \text{otherwise}, \end{cases} \tag{6}$$

where $N'_k = \sum_{i'=1}^{i-1} I(z_{i'} = k)$ is the number of previous observations from z_1 to z_{i-1} assigned to mixture k.

We place this non-parametric assumption on the mixture distribution because we expect there will be an infinite number of mixtures as we observe an infinite amount of data, but the DPMM will adaptively instantiate a finite number of mixtures for a finite number of observations.

F. Importance Sampling and Sequential Monte Carlo

A central issue in Bayesian inference is the computation of the often intractable integral $\bar{f} = \int f(\theta)P(\theta)d\theta$. For this task, importance sampling (IS) is a popular tool. To perform importance sampling, J samples of $\theta^{(j)}$ (also known as "particles")

are drawn from a proposal distribution $Q(\theta)$ that is simple to sample from and approximates $P(\theta)$ well. The integral is estimated using the weighted empirical average of these samples $f(\theta^{(j)})$ with weights $w^{(j)} = P(\theta^{(j)})/Q(\theta^{(j)})$ to approximate the integral:

$$\int f(\theta)P(\theta)d\theta \approx \sum_{j=1}^{J} w^{(j)} f\left(\theta^{(j)}\right). \tag{7}$$

For problems where θ arrives in a sequence, $\boldsymbol{\theta}_{1:N} = (\theta_1, \ldots, \theta_N)$, we can rewrite $w^{(j)}$ as a sequential update of the importance weight:

$$w_i^{(j)} = w_{i-1}^{(j)} \frac{P(\boldsymbol{\theta}_{1:i})}{P(\boldsymbol{\theta}_{1:i-1})Q(\theta_i|\boldsymbol{\theta}_{1:i-1})}. \tag{8}$$

In a Bayesian context, one can view this sequential importance sampler as sequentially updating a prior distribution at $i = 0$ to the posterior of all observed data at time N. Moving from $i = 1$ to N in this sampler may lead to a situation where the weight of one particle dominates the others, thereby increasing the variance of our Monte Carlo estimate and propagating suboptimal particles to future time steps. To avoid this, we can resample the particles according to their weights $w^{(j)}$ for $j = 1, \ldots, J$ particles–leading to the sequential Monte Carlo (SMC) sampler [34]. Monte Carlo techniques like IS and SMC are appealing from a computational perspective as we can trivially parallelize the computation for each weight $w_i^{(j)}$ to as many processors available as each weight is independent of the other.

We can sequentially update a Dirichlet process mixture model for online inference using an SMC sampler under general proposal distributions and modeling assumptions [35]. In the particle filtering algorithm for DP mixtures, we have an analytically convenient expression for the weight updates in the special case where the observation model is a Dirichlet process of normal inverse-Wisharts [36]. Moreover, SMC methods have also been used in online GP learning for sequentially updating GP hyperparameters, but rely on strong modeling and conjugate prior choices as this SMC sampler takes advantage of closed-form marginalization of nuisance parameters for efficient inference [37]. Later work has generalized the SMC sampler for updating the GP hyperparameters sequentially without restrictive conjugacy requirements [38]. However, these methods are not truly online methods and are unsuitable for large-scale streaming scenarios because the complexity of updating the particle weight still grows cubically with respect to the number of observations.

Finally, IS-MOE is a fast method that unifies non-stationary function learning and parallel GP inference [2]. To achieve scalability, IS-MOE integrates over the space of partitions using importance sampling, which allows distributed computation. This method uses "minibatched" stochastic approximations, where the model is fit with only a subset of the training set and the likelihood is upweighted to approximate the full data likelihood. However, IS-MOE cannot update GP models as new data arrive. For this purpose, we develop a new approach for online, parallelizable inference of a mixture-of-experts GP model that we call "GP-MOE".

G. Gaussian Processes and Optimization: A Bandit Perspective

Bayesian optimization techniques seek to find parameters that best model a conditional probability. Many approaches optimize parameter configurations adaptively [39], [40], [41], with bandit formulations being particularly successful in GP contexts [42], [43], [44]. The bandit framework considers the problem of optimizing a function f, sampled from a GP, by sequentially selecting from a set of arms corresponding to inputs $\mathbf{x}_i$, where noisy values y_i are observed.

The goal of a bandit algorithm is to sequentially select arms that maximize long-term rewards; to do this, one needs to approximate the expected rewards for each arm (exploration) and then select the arms that maximize rewards (exploitation). There are two common algorithmic strategies for choosing arms: optimization based algorithms and sampling based approaches. Here, we look at the context of using GP-distributed functions for bandit optimization. At time i, we select a new point $\mathbf{x}_i$ that maximizes some acquisition function. We can select the next point to evaluate in the function by optimizing with respect to the predictive reward:

$$\mathbf{x}_i := \underset{\mathbf{x}^* \in \mathbb{R}^D}{\operatorname{argmax}}\, \mathbf{m}(\mathbf{x}^*), \tag{9}$$

which can quickly converge to a poor local maxima. To better explore the function, we can select $\mathbf{x}_i$ with respect to the GP upper confidence bound (GP-UCB):

$$\mathbf{x}_i := \underset{\mathbf{x}^* \in \mathbb{R}^D}{\operatorname{argmax}}\, \mathbf{m}(\mathbf{x}^*) + \sqrt{\beta_i \cdot \mathbf{s}(\mathbf{x}^*)}, \tag{10}$$

where $\mathbf{m}(\cdot), \mathbf{s}(\cdot)$ are the predictive posterior mean and variance for a test point $\mathbf{x}^*$, and β_i is a tuning parameter that acts as the trade-off between exploration and exploitation. The GP-UCB acquisition function prefers to select new points where there is both high uncertainty and a high predicted reward [43].

In Thompson sampling (TS), we take samples from the posterior predictive distribution evaluated at $\mathbf{X}^*$ and select the test input that yields the highest sample of y^*:

$$\begin{aligned}(y_1^*, \ldots, y_{N_{samp}}^*) &\sim P(\mathbf{y}^*|\mathbf{X}, \mathbf{X}^*, \mathbf{y}),\\ \mathbf{x}_i &:= \mathbf{x}_{\operatorname{argmax}_{i^*}(y_1^*, \ldots, y_{N_{samp}}^*)}.\end{aligned} \tag{11}$$

The TS approach to the bandit problem is closely related to optimizing the acquisition function with respect to the UCB bound and can be seen as a sampling-based variant balancing the exploration and exploitation trade-off [45].

Online GPs are a naturally appealing method for Bayesian optimization. The function being optimized, f, is usually difficult to evaluate. Hence, we are unlikely to observe more than handful of evaluations of f upon initially fitting the GP model, and we are unlikely to have an accurate estimate of the hyperparameters at the initial fit of the model. Since we do not know the GP hyperparameters of Bayesian optimization a priori either, we should update the GP as new function queries arrive.

III. Sequential Gaussian Processes for Online Learning

We assume that our data is generated from a Gaussian process mixture, similar to previous mixture-of-expert models for Gaussian processes. This hierarchical model allows for greater flexibility in modeling functions, at the cost of more difficult inference for which we propose a distributable solution in the next section. Our approach adopts the following generative model:

$$\begin{aligned}\mathbf{x}_i &\sim \mathcal{T}(\boldsymbol{\mu}_{z_i}, \boldsymbol{\Psi}_{z_i}, \nu_{z_i}), \quad \alpha \sim \text{Gamma}(a_0, b_0),\\ z_i|\alpha &\sim \text{CRP}(\alpha), \quad (\theta_k, \sigma_k^2) \sim \log\mathcal{N}(\mathbf{m}_0, s_0^2\mathbf{I}),\\ \mathbf{y}_k|\mathbf{X}_k, \theta_k, \sigma_k^2 &\sim \mathcal{N}(0, \boldsymbol{\Sigma}_{\theta_k} + \sigma_k^2\mathbf{I}),\end{aligned} \tag{12}$$

where $(\mathbf{X}_k, \mathbf{y}_k) = (\mathbf{x}_i, y_i : z_i = k)$ represent the data associated with the mixture k. We assume that the inputs are distributed according to a Dirichlet process mixture of normal-inverse Wishart distributions, and we marginalize out the parameter locations from the inputs. The outputs are then assumed to be generated by independent GPs, given the mixture indicator. The GP parameters, $\boldsymbol{\theta}_k = [\theta_k, \sigma_k^2]$, are assumed to be log-normally distributed, and we update the state of $\boldsymbol{\theta}_k$ using the elliptical slice sampler [46].

The elliptical slice sampler (ESS) is an MCMC technique for sampling from a posterior distribution where the prior is assumed to be a multivariate Gaussian distribution but the likelihood is not conjugate. For the ESS algorithm, the sampler is always guaranteed to accept a transition to a new state unlike typical Metropolis-Hastings samplers and therefore is very efficient for MCMC sampling. In fact, slice sampling techniques are preferable in posterior sampling of the covariance function hyperparameters in GP models [47].

We assign the i-th input sequentially to clusters according to the Chinese restaurant prior and the mixture locations marginalized out [48]:

$$P(z_i = k|\alpha, \mathbf{X}_k) \propto \begin{cases} N'_k \cdot \mathcal{T}(\boldsymbol{\mu}'_k, \boldsymbol{\Psi}'_k, \nu'_k) & k \in K^+. \\ \alpha \cdot \mathcal{T}(\boldsymbol{\mu}_0, \boldsymbol{\Psi}_0, \nu_0) & \text{o.w.} \end{cases} \tag{13}$$

where $\mathcal{T}(\boldsymbol{\mu}'_k, \boldsymbol{\Psi}'_k, \nu'_k)$ is a multivariate-t distribution with the mean, covariance, and degrees-of-freedom parameters for observation i's sequential assignment to mixture k, where K^+ refers to the previously occupied clusters, $\{k : N'_k > 0\}$.

$$\begin{aligned}\boldsymbol{\mu}'_k &= \frac{\lambda_0\boldsymbol{\mu}_0 + N'_k\bar{\mathbf{x}}_k}{\lambda'_k}, \bar{\mathbf{x}}'_k = \frac{\sum_{i':(z_{i'}=k, i'<i)} \mathbf{x}_{i'}}{N'_k},\\ N'_k &= \sum_{i'=1}^{i-1} I(z_{i'} = k), \lambda'_k = \lambda_0 + N'_k,\\ \nu'_k &= \nu_0 + N'_k - D + 1,\\ \boldsymbol{\Psi}'_k &= \frac{\lambda'_k + 1}{\lambda'_k\nu'_k}\left(\boldsymbol{\Psi}_0 + \mathbf{S}'_k + \mathbf{S}'_{\bar{\mathbf{x}}_k}\right)\\ \mathbf{S}'_k &= \sum_{i':(z_{i'}=k, i'<i)} (\mathbf{x}_{i'} - \bar{\mathbf{x}}'_k)(\mathbf{x}_{i'} - \bar{\mathbf{x}}'_k)^T\end{aligned}$$

$$\mathbf{S}'_{\bar{\mathbf{x}}_k} = \frac{\lambda_0 N'_k}{\lambda'_k}(\bar{\mathbf{x}}'_k - \boldsymbol{\mu}_0)(\bar{\mathbf{x}}'_k - \boldsymbol{\mu}_0)^T. \tag{14}$$

We use the $(\cdot)'$ notation to indicate that the summary statistics are conditioned only on observations $i' = 1, \ldots, i-1$. We also place a Gamma prior on the DP concentration parameter, α, which allows us to easily sample its full conditional up to observation i with a variable augmentation scheme [49]:

$$\begin{aligned}\rho|\alpha &\sim \text{Beta}(\alpha + 1, i), K = |\{k : N_k > 0\}|\\ \frac{\pi_\alpha}{1 - \pi_\alpha} &= \frac{a_0 + K - 1}{N(b_0 - \log\rho)}\\ \alpha|\mathbf{z}_{1:i}, \pi_\alpha, \rho &= (1 - \pi_\alpha) \cdot \text{Gamma}(\alpha_0 + K - 1, b_0 - \log\rho)\\ &\quad + \pi_\alpha \cdot \text{Gamma}(\alpha_0 + K, b_0 - \log\rho).\end{aligned} \tag{15}$$

A. SMC for Online GP-MOE

In an SMC setting with $j = 1, \ldots, J$ particles, we first propagate the particles $(\mathbf{z}^{(j)}, \boldsymbol{\theta}^{(j)}, \alpha^{(j)})$ from time $i-1$ to i and fit a GP product-of-experts model. Then we calculate the particle weights. At the initial time, $i = 1$, the particle weight j is:

$$w_1^{(j)} \propto P(y_1|z_1^{(j)}, \mathbf{x}_1, \boldsymbol{\theta}^{(j)})P(\mathbf{x}_1|z_1^{(j)}, \alpha^{(j)}). \tag{16}$$

For $i > 1$, the particle weight in (8) can written as the product of:

1) The previous weight, $w_{i-1}^{(j)}$,
2) The ratio of the model's likelihood up to observation i over the likelihood up to observation $i-1$ [38],
3) The particle weight of $z_i^{(j)}$ for the Dirichlet process mixture model [36].

The GP term of the particle weight from [38] in this setting simplifies to the ratio of the new likelihood (including observation i) and the old likelihood (excluding observation i) of the mixture z_i. We can store the old likelihood in memory from the last time we updated the particle weights, so the only computationally intensive step is computing the new likelihood for mixture z_i. This particle update is:

$$w_i^{(j)} := w_{i-1}^{(j)} \cdot \frac{P\left(\mathbf{y}_{z_i^{(j)}} \middle| \mathbf{X}_{z_i^{(j)}}, \boldsymbol{\theta}_{z_i^{(j)}}^{(j)}\right)}{P\left(\mathbf{y}'_{z_i^{(j)}} \middle| \mathbf{X}'_{z_i^{(j)}}, \boldsymbol{\theta}'^{(j)}_{z_i^{(j)}}\right)} \cdot P(\mathbf{x}_i|z_i^{(j)}, \alpha^{(j)}) \tag{17}$$

where

$$\begin{aligned}(\mathbf{X}'_k, \mathbf{y}'_k) &= (\mathbf{x}_{i'}, y_{i'}, i' : (z_{i'} = k, i' < i)),\\ (\mathbf{X}_k, \mathbf{y}_k) &= (\mathbf{x}_i, y_i, i : z_i = k)\end{aligned} \tag{18}$$

After calculating the particle weights, we calculate the effective sample size, $N_{\text{eff}} = 1/\sum_{j=1}^J (w_i^{(j)})^2$. If the effective number of samples drops below a certain threshold (typically $J/2$), then we resample the particles with probability $w_i^{(1)}, \ldots, w_i^{(J)}$ to avoid the particle degeneracy problem. The details for updating the particles are in Algorithm 1.

To calculate the predictive posterior distribution of the GP-MOE for test data $\mathbf{x}_*$, we calculate the predictive mean and variance on each individual particle, averaged over the mixture

Algorithm 1: Online GP-MOE.

Input: New observation, $(\mathbf{x}_i, y_i)$.

/* Particle propagation. */

for $j = 1, \ldots, J$ *in parallel* **do**

Sample $z_i^{(j)}$ from $P(z_i^{(j)}|\alpha^{(j)}, \mathbf{X}_{1:i-1})$, in Eq. 13

Sample $\boldsymbol{\theta}_{z_i}^{(j)}$ using the elliptical slice sampler from [47].

Sample $\alpha^{(j)}$ from the full conditional $P(\alpha^{(j)}|\mathbf{z}_{1:i})$ in Eq. 15.

Update particle weight $w_i^{(j)}$ using Eq 17.

Normalize weights, $w_i^{(j)} := w_i^{(j)} / \sum_{j=1}^{J} w_i^{(j)}$.

/* Particle resampling. */

if $N_{eff} < J/2$ **then**

Resample particles $\left(\mathbf{z}_{1:i}^{(\mathbf{j}^*)}, \boldsymbol{\theta}^{(\mathbf{j}^*)}, \alpha^{(\mathbf{j}^*)}\right)$ from $\mathbf{j}^* \sim$ Multinomial $\left(J, w_i^{(1)}, \ldots, w_i^{(J)}\right)$.

Set $w_i^{(j)} := 1/J$ for $j = 1, \ldots, J$.

Output: Particle weights $\left(w_i^{(1)}, \ldots, w_i^{(J)}\right)$ and particles $\left(\mathbf{z}_{1:t}^{(1:J)}, \boldsymbol{\theta}^{(1:J)}, \alpha^{(1:J)}\right)$.

Algorithm 2: GP-MOE Prediction.

for $j = 1, \ldots, J$ *in parallel* **do**

Predict new observations on particle j with

$$p_k \propto N_k \cdot P(\mathbf{x}_*|z_* = k, \mathbf{X}_k, -)$$

$$P(y_*^{(j)}|\mathbf{y}, \mathbf{X}, \mathbf{x}_*, -) = \sum_{k \in K^+} p_k \cdot P(y_{k*}^{(j)}|\mathbf{y}_k, \mathbf{X}_k, \mathbf{x}_*, -)$$

Average predictions:

$P(\bar{y}_*|\mathbf{y}, \mathbf{X}, \mathbf{x}_*) = \sum_{j=1}^{J} w_i^{(j)} P(y_*^{(j)}|\mathbf{y}, \mathbf{X}, -)$

assignment for the test data. Then, we average the predictive distribution on each particle, weighted by $w_i^{(1)}, \ldots, w_i^{(J)}$. The details for prediction in GP-MOE are available in Algorithm 2.

Assuming each batch is on average size N/K, the computational complexity of fitting our model is $\mathcal{O}(JN^3/K^2)$ for J particles. SMC methods allows for parallel computation, as we can update the particles independently. Thus, we can reduce the computational complexity to fitting GP-MOE to be $\mathcal{O}(N^3/K^2)$. While the computational complexity of GP-MOE is considerably reduced from the typical $\mathcal{O}(N^3)$ cost, the complexity still grows considerably as new data arrive.

To mitigate this problem, we can adopt an approach where we subsample $B \ll N_k$ observations uniformly without replacement within a mixture assignment to approximate the likelihood $P(\mathbf{y}_k|\mathbf{X}_k, \boldsymbol{\theta}_k)$ by upweighting the likelihood by a power of N_k/B. This stochastic approximation provides an estimate for the full likelihood and has been used successfully in other Bayesian inference algorithms [50], [51].

$$\begin{aligned}
&\mathbf{u}_k^{(j)} = (u_1, \ldots, u_B) \sim \text{HyperGeometric}\left(B, \left\{i : z_i^{(j)} = k\right\}\right),\\
&(\mathbf{y}_{\mathbf{u}_k}, \mathbf{X}_{\mathbf{u}_k}) = \left(y_u, \mathbf{x}_u : u \in \mathbf{u}_k^{(j)}\right),\\
&P\left(\mathbf{y}_{\mathbf{u}_k^{(j)}}|\mathbf{X}_{\mathbf{u}_k^{(j)}}, \boldsymbol{\theta}_k^{(j)}\right) \sim \mathcal{N}\left(0, \boldsymbol{\Sigma}_{\theta_k^{(j)}} + \frac{N_k \sigma_k^2}{B}\mathbf{I}\right). \qquad (19)
\end{aligned}$$

TABLE I
COMPARISON OF INFERENCE COMPLEXITY. N IS THE NUMBER OF DATA POINTS, K IS THE NUMBER OF EXPERTS, M IS THE NUMBER OF INDUCING POINTS, AND J IS THE NUMBER OF PARTICLES

GP	Sparse GP	WISKI
$\mathcal{O}(N^3)$	$\mathcal{O}(NM^2)$	$\mathcal{O}(M \log M)$
POE	**GP-MOE**	**Minibatched GP-MOE**
$\mathcal{O}(N^3/K^2)$	$\mathcal{O}(JN^3/K^2)$	$\mathcal{O}(J \min\{N_k, B\}^3/K^2)$

We use this stochastic approximation when the number of observations in a mixture, $N_k^{(j)}$ exceeds B, so that the complexity of fitting the GP-MOE does not increase when $N_k^{(j)} > B$. With this stochastic approximation, the complexity of our model is $\mathcal{O}(J \min\{N_k, B\}^3/K^2)$. In Table I, we compare the computational complexity of our GP-MOE method, POE (our method using only one particle), WISKI [22] and OSVGP [18].

B. Optimization in a Bandit Setting With Online GP-MOE

To motivate using the online GP-MOE for an applied problem, we show how this approach can be useful for a typical setting where GPs are popularly used–optimization. We can perform bandit Gaussian process optimization using our online GP-MOE method. The objective of this setting is that we want to optimize the function, f, using a mixture of GPs, where the arms of the bandit are indexed by k, corresponding to the kernel-specific hyperparameters of the mixture components.

We update the model by first selecting a new test point $\mathbf{x}_i$ using the UCB Thompson-Sampling from (11). Then, we evaluate the point $\mathbf{x}_i$ at $f(\mathbf{x}_i)$ and update the model using Algorithm 1 at $(\mathbf{x}_i, f(\mathbf{x}_i))$. $\mathbf{x}_i$ stochastically selects the arm, z_i, which produces the best reward that corresponds to the marginal likelihood $P(\mathbf{y}_k|\mathbf{X}_k, \boldsymbol{\theta}_k, \mathbf{z}_{1:i})$. Lastly, we can update the particle weight, $w_i^{(j)}$, and the other model parameters, $\boldsymbol{\theta}_k^{(j)}$ and $\alpha^{(j)}$ after selecting the arm. We continue this procedure for N total function evaluations.

TABLE II
ONLINE PREDICTIVE LOG LIKELIHOOD OVER FIVE TRIALS. ONE STANDARD ERROR REPORTED IN PARENTHESES

	Motorcycle	Nile	Brent	Canada	EUR-USD	Heart Rate
GP-MOE 500	**-63.686 (2.370)**	-144.397 (2.447)	**266.563 (45.322)**	**305.852 (14.763)**	-4585.758 (20.097)	**-8328.856 (31.848)**
GP-MOE 100	-72.157 (3.841)	-147.531 (4.369)	120.428 (73.548)	261.257 (37.422)	-4539.573 (42.731)	-8790.210 (32.288)
POE	-114.665 (11.187)	-142.832 (1.513)	-456.628 (17.927)	-114.958 (12.374)	-4538.173 (25.029)	-12023.802 (158.452)
WISKI	-112.467 (0.000)	-127.998 (0.000)	-800.852 (0.000)	-152.242 (0.000)	**-4482.185 (0.000)**	-10261.351 (0.000)
OSVGP 500	-99.523 (3.019)	**-127.289 (0.157)**	-1250.734 (142.780)	43.021 (5.292)	-4766.513 (43.271)	-14307.737 (278.164)
OSVGP 100	-125.862 (0.306)	-138.537 (0.057)	-731.435 (156.499)	-230.525 (0.893)	-4494.476 (0.244)	-14550.117 (107.419)
OSVGP 1	-135.776 (0.100)	-145.048 (0.116)	-1438.428 (3.493)	-313.022 (0.296)	-4676.530 (5.742)	-13586.578 (145.657)

TABLE III
ONLINE PREDICTIVE MEAN SQUARED ERROR OVER FIVE TRIALS. ONE STANDARD ERROR REPORTED IN PARENTHESES

	Motorcycle	Nile	Brent	Canada	EUR-USD	Heart Rate
GP-MOE 500	**0.389 (0.007)**	**0.722 (0.003)**	**0.049 (0.006)**	0.019 (0.002)	1.010 (0.002)	**0.379 (0.002)**
GP-MOE 100	0.417 (0.019)	0.740 (0.005)	0.061 (0.008)	**0.018 (0.002)**	1.006 (0.002)	0.438 (0.002)
POE	0.479 (0.038)	0.807 (0.017)	0.123 (0.007)	0.102 (0.019)	1.016 (0.002)	0.677 (0.006)
WISKI	0.631 (0.000)	0.767 (0.000)	0.177 (0.000)	0.048 (0.000)	1.007 (0.000)	0.408 (0.000)
OSVGP 500	0.413 (0.028)	0.765 (0.003)	0.922 (0.047)	0.028 (0.001)	1.036 (0.013)	0.939 (0.011)
OSVGP 100	0.802 (0.004)	0.852 (0.001)	0.444 (0.050)	0.366 (0.004)	**1.003 (0.000)**	0.935 (0.005)
OSVGP 1	0.986 (0.001)	0.934 (0.001)	0.916 (0.003)	0.929 (0.005)	1.017 (0.002)	0.812 (0.016)

Algorithm 3: GP-MOE Bandit Optimization with Thompson Sampling.

for $i = 1, \ldots, N$ **do**
  Sample points with $(y_1^*, \ldots, y_{N^*}^*) \sim P(\mathbf{y}^*|\mathbf{X}, \mathbf{X}^*, \mathbf{y})$ using Alg. 2.
  Select new point , $\mathbf{x}_i := \mathbf{x}_{\text{argmax}_{i^*}(y_1^*, \ldots, y_{N^*}^*)}$.
  Update model with $(\mathbf{x}_i, f(\mathbf{x}_i))$ using Alg. 1.

IV. EMPIRICAL ANALYSES FOR ONLINE GP-MOE

To demonstrate the ability of our algorithm to fit streaming non-stationary GPs, we apply our online GP-MOE to a collection of empirical time-series datasets that exhibit non-stationary behavior.[1] In our comparisons, we look at the following datasets: 1.) An accelerometer measurement of a motorcycle crash. 2.) The price of Brent crude oil. 3.) The annual water level of the Nile river data. 4.) The exchange rate between the Euro and the US Dollar. 5.) The annual carbon dioxide output in Canada. 6.) The heart rate of a hospital patient in the MIMIC-III data set [52]. We pre-process the data so that the inputs and outputs have zero-mean and unit variance.

We compare our method against three alternative approaches: a product-of-experts model (POE), which is a special case of our algorithm with only one particle, a sparse online GP method using the Woodbury identity and structured kernel interpolation (WISKI) [22], and an online sparse variational GP method[2] (OSVGP) [18]. Our choice of kernel for each of these methods is the radial basis function (RBF):

$$\Sigma_\theta(\mathbf{x}, \mathbf{x}') = \exp\left\{-\frac{\theta}{2}\sum_{d=1}^{D}(x_d - x_d')^2\right\}, \tag{20}$$

for length scale parameter, θ.

In these experiments, we set the number of inducing points to be 50 for each of the sparse methods. We evaluate our method when J is equal to 1 (equivalent to a POE), 100, and 500 particles. The OSVGP method requires a fixed number of optimization iterations to estimate the variational parameters, and the results are highly sensitive to this setting. To make the model settings comparable to the GP-MOE settings, we also set the optimizer iterations to 1, 100, and 500. In the GP-MOE model, we distribute the inference of each particle over 16 cores on a shared memory process using OpenMP. In this setting, we run an online prediction experiment where we initialize the model using the first observation in the time-series data set. Then, we sequentially predict the subsequent observation and update the model with the next data point.

Our method generally performs better than the competing online GP methods (Tables II and III). We broadly observe that the GP-MOE performs better than POE because we can integrate over the space of partitions and, thus, we will better capture the predictive uncertainty. However, WISKI is undoubtedly the fastest method as the computational complexity is constant with respect to the number of observations. But because these data sets exhibit non-stationarity, WISKI is not able to handle changes in the kernel behavior (like heteroscedasticity, for example) and therefore performs poorly in terms of MSE and log likelihood in these experiments. The GP-MOE methods perform comparatively to the OSVGP, which only uses a single GP, in terms of wall time (Table IV).

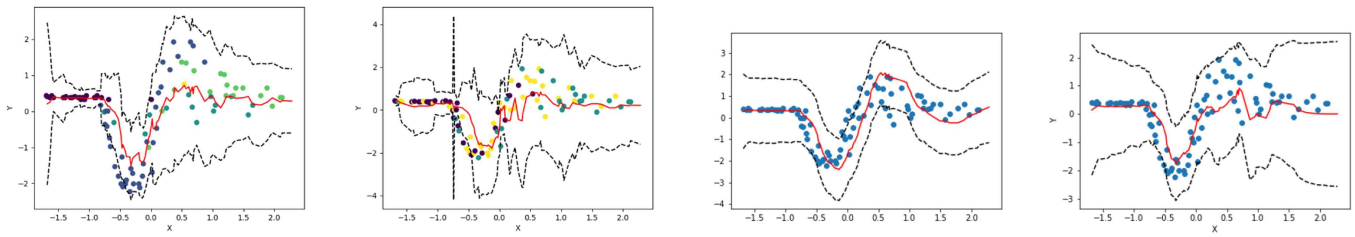
The performance of the OSVGP strongly depends on the number of optimizer iterations, and OSVGP has poor performance when only one iteration is used as opposed to 500 iterations. OSVGP often runs into numerical stability issues

[1]The motorcycle dataset is available in the R package `VarReg`. The Brent, Canada CO_2, and Nile datasets are available here: https://github.com/alan-turing-institute/TCPD. The EUR-USD dataset is available in the R package `priceR`.

[2]The implementation for OSVGP and WISKI is available here: https://github.com/wjmaddox/online_gp. Our code is available at https://github.com/michaelzhang01/GPMOE.

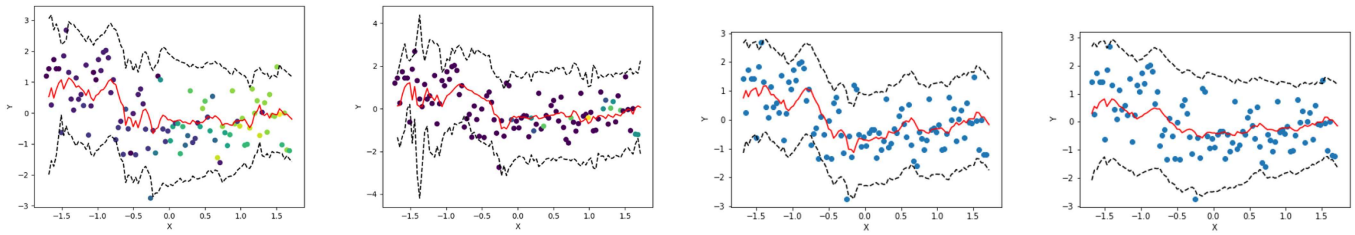
TABLE IV
CPU WALL TIME IN SECONDS FOR ONLINE PREDICTION OVER FIVE TRIALS. ONE STANDARD ERROR REPORTED IN PARENTHESES

	Motorcycle	Nile	Brent	Canada	EUR-USD	Heart Rate
GP-MOE 500	608.442 (5.507)	235.979 (6.036)	12503.964 (1133.177)	2956.015 (112.812)	67034.274 (15869.414)	34015.384 (286.470)
GP-MOE 100	203.195 (2.664)	69.393 (0.808)	4505.307 (508.852)	1077.938 (39.106)	11200.608 (2385.012)	8896.295 (20.048)
POE	85.524 (1.488)	28.604 (0.529)	1053.192 (98.926)	324.706 (19.900)	1597.147 (398.397)	1094.152 (9.027)
WISKI	**1.772 (0.363)**	**1.347 (0.095)**	**16.297 (3.233)**	**3.429 (0.170)**	**50.901 (6.921)**	463.025 (33.449)
OSVGP 500	1693.370 (182.273)	1648.170 (99.036)	19176.419 (2168.529)	3908.914 (239.521)	57412.117 (1644.124)	193332.795 (14744.607)
OSVGP 100	323.504 (74.718)	347.613 (40.670)	3819.830 (350.382)	830.059 (118.029)	10829.119 (1571.008)	41138.089 (2890.219)
OSVGP 1	2.729 (0.631)	3.156 (0.543)	33.614 (3.736)	5.929 (0.254)	83.713 (10.725)	**442.911 (22.729)**



(a) GP-MOE (b) POE (c) WISKI (d) OSVGP

Fig. 1. Online posterior predictive mean (plotted with solid red lines) and 95% credible intervals (plotted with dashed black lines) for the motorcycle dataset. The color of the data points in these figures for GP-MOE and POE represent the mixture assignment of that observation for the particle with the highest weight. $N = 94$.



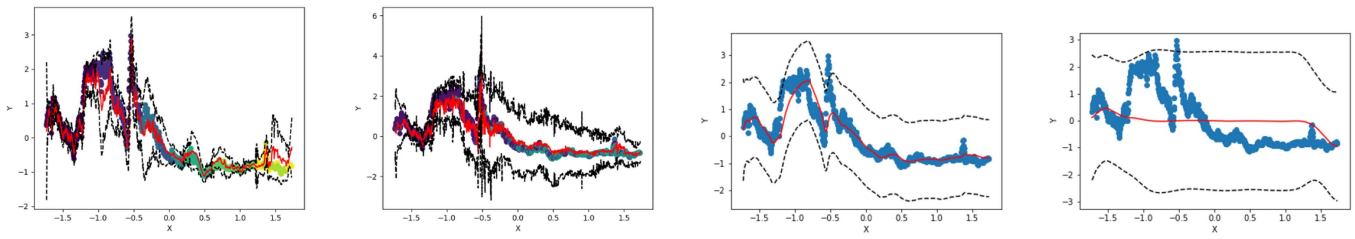
(a) GP-MOE (b) POE (c) WISKI (d) OSVGP

Fig. 2. Online posterior predictive mean (plotted with solid red lines) and 95% credible intervals (plotted with dashed black lines) for the Nile river dataset. The color of the data points in these figures for GP-MOE and POE represent the mixture assignment of that observation for the particle with the highest weight. $N = 100$.

relating to calculating Cholesky decompositions in the sparse variational GP [22]. Like WISKI, the OSVGP can sometimes obtain adequate point estimates for the posterior mean but generally mischaracterizes the posterior predictive variance in these non-stationary examples. While OSVGP can somewhat capture the non-stationary behavior as the hyperparameters update as new data arrive, the posterior predictive variance is generally wide compared to GP-MOE as evidenced by the superior performance of GP-MOE with respect to the predictive log likelihood.

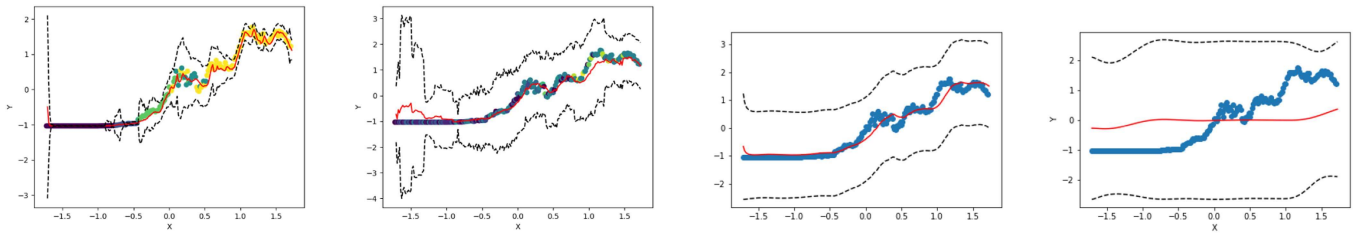
For the motorcycle, Brent, Canadian carbon dioxide, and heart rate datasets, the GP-MOE performs the best in terms of predictive mean squared error and log likelihood. In the Nile river data set, the OSVGP performs the best in terms of online predictive log likelihood. This could be because the only non-stationary component of the Nile river data set is the mean value, which we assume to be constant at all values of $\mathbf{x}$ in GP-MOE. The OSVGP and WISKI obtain the best MSE and log likelihood results on the EUR-USD data sets as well, which exhibits only time varying noise, zero mean, and stationary length-scale for the entire duration of the time-series data. Here, OSVGP and WISKI produce wider noise estimates than GP-MOE. However, a stochastic volatility model would perhaps be a better fit for this type of data than a mixture of GPs. The other datasets exhibit time-varying noise terms and length scales, which our mixture of GPs can capture, and thus the GP-MOE exhibits superior performance.

The online predictive performance of each of these online GP models show that WISKI and OSVGP produce overly wide credible intervals in comparison to GP-MOE and POE (Figs. 1–5). In some instances, WISKI and OSVGP produce inaccurate predictive means in the case of the motorcycle, Brent, and Canada data sets. In the GP-MOE and POE results, we have colored the observations based on the mixture assignments of the largest particle. We can see from these plots that the mixture assignments largely correspond to changes in the underlying functional behavior. For example in the plot for the motorcycle data set, we observe an initial low-noise regime and a subsequent high-noise regime. The GP-MOE assigns data to different mixtures in these separate parts of the time series where the noise and



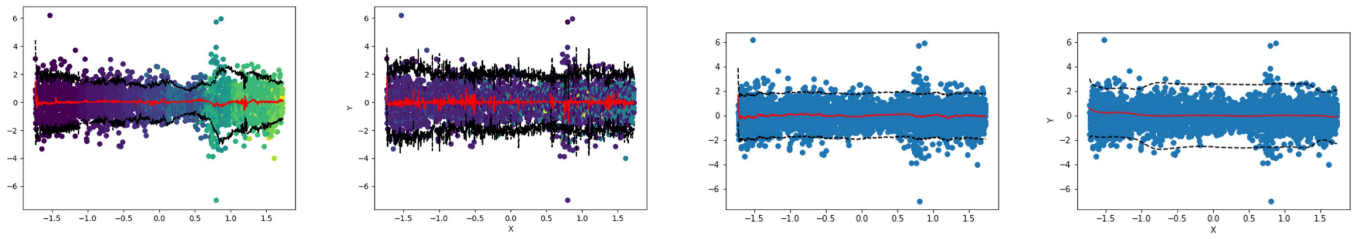
(a) GP-MOE (b) POE (c) WISKI (d) OSVGP

Fig. 3. Online posterior predictive mean (plotted with solid red lines) and 95% credible intervals (plotted with dashed black lines) for the Brent crude oil dataset. The color of the data points in these figures for GP-MOE and POE represent the mixture assignment of that observation for the particle with the highest weight. $N = 1025$.



(a) GP-MOE (b) POE (c) WISKI (d) OSVGP

Fig. 4. Online posterior predictive mean (plotted with solid red lines) and 95% credible intervals (plotted with dashed black lines) for the Canadian CO_2 dataset. The color of the data points in these figures for GP-MOE and POE represent the mixture assignment of that observation for the particle with the highest weight. $N = 215$.



(a) GP-MOE (b) POE (c) WISKI (d) OSVGP

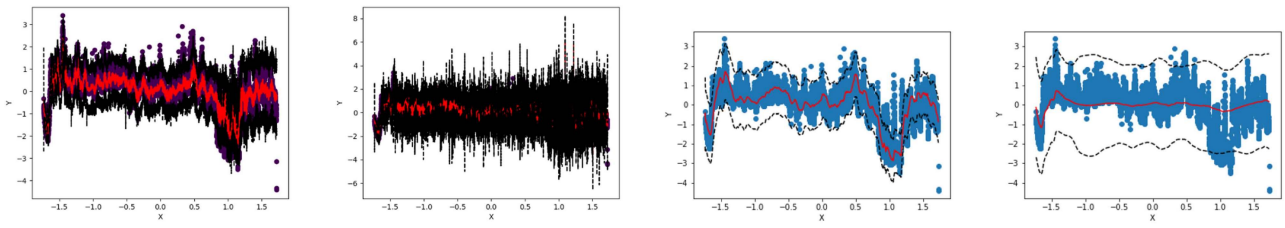
Fig. 5. Online posterior predictive mean (plotted with solid red lines) and 95% credible intervals (plotted with dashed black lines) for the USD-EUR exchange rate dataset. The color of the data points in these figures for GP-MOE and POE represent the mixture assignment of that observation for the particle with the highest weight. $N = 3139$.

length scale behavior changes (Figs. 1–5). In contrast, stationary methods like WISKI or OSVGP model the motorcycle data with a constant noise term across time, and we can see that the initial low-noise regime of the data is modeled with an extremely large predictive 95% credible interval.

A. Hyperparameter Settings

Our approach has important hyperparameters that can affect the performance of our online GP-MOE algorithm. Specifically, these important hyperparameters are the number of particles and the minibatch size. The authors of the IS-MOE method provides an in-depth examination of how changing these hyperparameters affect the predictive performance in terms of test set log likelihood and mean squared error [2]. Increasing the number of importance samples, J, and minibatch size B of the algorithm will improve model performance and increase computation time, while increasing the number of experts, K, will decrease both computation time and model performance (as K increases, the covariance matrix becomes diagonally dominant, and is unable to capture input correlation). However, there are intermediate settings for each parameter that can drastically reduce computation time without a decrease in predictive model performance.

We can look at the effect of increasing J and B on the online predictive MSE and log likelihood (Fig. 7). Unsurprisingly, increasing J and B will improve the predictive performance of GP-MOE. However, there is a point at which the performance of GP-MOE will plateau with respect to an increasing J and B, indicating that there are lower settings for which we can obtain accurate performance.



(a) GP-MOE (b) POE (c) WISKI (d) OSVGP

Fig. 6. Online posterior predictive mean (plotted with solid red lines) and 95% credible intervals (plotted with dashed black lines) for the heart rate dataset. The color of the data points in these figures for GP-MOE and POE represent the mixture assignment of that observation for the particle with the highest weight. $N = 10000$.

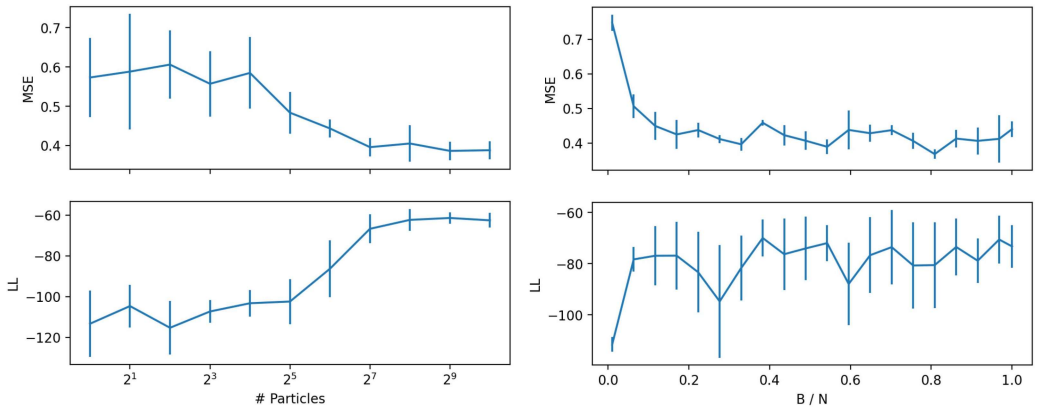
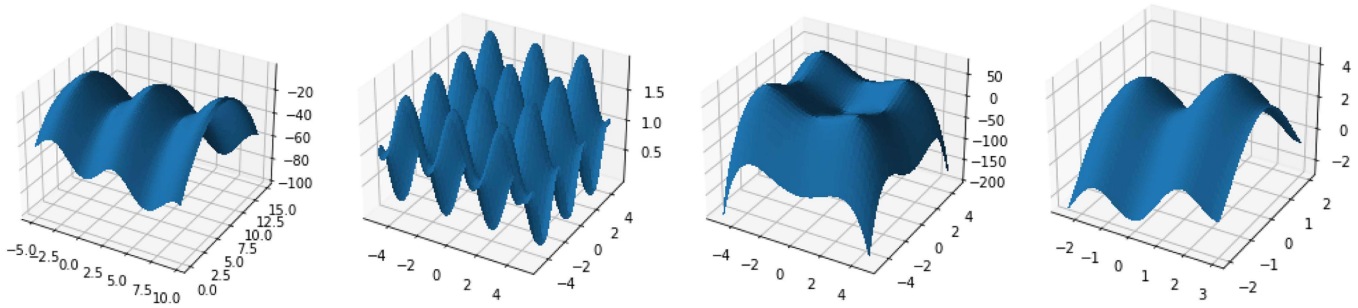


Fig. 7. Left: Performance of GP-MOE with increasing values of J on the motorcycle dataset. Right: Performance of GP-MOE with increasing values of B on the motorcycle dataset.



(a) Branin-Hoo (b) Parsopoulos (c) Styblinski-Tang (d) Ursem

Fig. 8. Test functions for the bandit optimization experiments.

B. Bandit Optimization

Next, we use our GP-MOE in a bandit optimization setting using Thompson sampling. Here, we are maximizing four functions typically used to evaluate optimization algorithms: the Branin-Hoo function, the Syblinski-Tang function, the Parsopoulos function, and the Ursem function (plotted in Fig. 8). We choose these functions as they are multimodal, and we want to examine how well the competing GP optimization methods both explore and exploit the target function.

We compared GP-MOE, POE, WISKI, OSVGP and GP-TS on these four optimization tasks, setting $J = 100$ for the GP-MOE, and the number of inducing points to be 50 for the sparse methods. We run each method for $N = 500$ iterations. In bandit optimization, we focus on two results: Minimizing the mean average regret (MAR) and obtaining the maximum value of the function $\max_{\mathbf{x}} f(\mathbf{x})$. GP-MOE and POE typically obtain the highest (best) maximum value, while the stationary methods (especially the basic GP-TS) obtain better MAR (Table V).

This suggests that basic online GP approaches are liable to only explore a suboptimal mode. While the bandit can reliably estimate the neighborhood surrounding this mode, thus resulting in a low MAR, it cannot actually find a better optimum compared to GP-MOE and POE. The only setting where a stationary method can compete with GP-MOE and POE is for the Parsopoulos function where GP-TS performs comparably

TABLE V
MEAN ABSOLUTE REGRET AND MAXIMUM FUNCTION VALUE FOR OPTIMIZATION EXPERIMENTS OVER FIVE TRIALS FOR 500 ITERATIONS. ONE STANDARD ERROR REPORTED IN PARENTHESES

	Branin-Hoo		Parsopoulos		Styblinski-Tang		Ursem	
	MAR	$\max f(\mathbf{x})$	MAR	$\max f(\mathbf{x})$	MAR	$\max f(\mathbf{x})$	MAR	$\max f(\mathbf{x})$
MOE	10.64 (5.54)	-0.41 (0.01)	**0.107 (0.005)**	**1.999 (0.001)**	5.65 (1.38)	**78.06 (0.19)**	0.63 (0.15)	6.336 (0.029)
POE	23.85 (2.14)	**-0.42 (0.02)**	0.202 (0.030)	**1.999 (0.001)**	12.84 (1.87)	78.05 (0.21)	0.98 (0.19)	**6.342 (0.034)**
WISKI	2.50 (0.12)	-19.86 (0.04)	0.418 (0.011)	1.996 (0.003)	0.418 (0.011)	1.996 (0.003)	0.42 (0.02)	5.338 (0.082)
OSVGP	56.68 (0.59)	-20.35 (0.30)	0.952 (0.030)	1.985 (0.009)	0.952 (0.030)	1.985 (0.009)	2.06 (0.07)	5.266 (0.056)
GP-TS	**0.96 (0.15)**	-19.93 (0.08)	0.351 (0.038)	**1.999 (0.001)**	**0.351 (0.038)**	1.999 (0.001)	**0.34 (0.01)**	5.374 (0.020)

TABLE VI
CPU WALL TIME FOR OPTIMIZATION EXPERIMENTS OVER FIVE TRIALS FOR 500 ITERATIONS. ONE STANDARD ERROR REPORTED IN PARENTHESES

	Branin-Hoo	Parsopoulos	Styblinski-Tang	Ursem
MOE	1100.41 (624.45)	775.88 (180.24)	1154.05 (738.10)	1278.11 (299.29)
POE	**21.53 (1.58)**	**25.78 (0.55)**	**21.88 (0.86)**	**22.25 (0.81)**
WISKI	51.90 (5.98)	44.78 (7.59)	56.00 (31.70)	48.95 (8.81)
OSVGP	1346.01 (60.04)	1216.63 (167.58)	1546.88 (974.74)	1227.35 (116.67)
GP-TS	1513.50 (187.90)	1161.05 (517.87)	949.81 (324.08)	779.65 (50.75)

to GP-MOE and POE. Though the POE can attain a maximum function value on par with GP-MOE, we see that the GP-MOE produces a maximum value with typically lower variance than the POE, which is an effect we would expect to see in an ensemble method like SMC. In these optimization experiments, we see that POE has the lowest CPU wall time. Thus, for an optimization task, like the ones analysed in this section, the optimal setting may be one where the number of particles, J, is rather low in order to take advantage of the variance minimization of SMC while maintaining the speed of the POE.

V. CONCLUSION AND FUTURE DIRECTIONS

In this paper, we introduced an online inference algorithm for fitting mixtures of Gaussian processes that can perform online estimation of non-stationary functions. We show that we can apply this mixture of GP experts in an optimization setting. In these bandit optimization experiments, we observe that GP-MOE finds the highest maximum value compared to the competing methods, and can generally produce better performance in terms of minimizing regret.

For future work, we are interested in extending this method in a few different domains: First, we want to implement additional features that may improve the performance of our online method with regards to the sequential Monte Carlo sampler. Among these, we want to investigate incorporating retrospective sampling of the mixture assignments, $\mathbf{z}$, as we only sample these indicators once when the i-th observation arrives. Retrospective sampling could lead to better predictive performance in subsequent observations, though the additional computational cost could be prohibitive for extremely large data sets.

Moreover, we are interested in introducing control variates in conjunction with the SMC sampler in order to reduce the variance of the Monte Carlo estimates. Basic Monte Carlo methods tend to exhibit high estimator variance. But, if we have some random variables correlated with our estimator whose expectation is known, then we can reduce the variance by a considerable amount. We seek to investigate these possible extensions in future work.

Next, we are interested in applying our online mixture of expert approach for modeling patient vital signs in a hospital setting. Gaussian processes have enjoyed notable success in the health-care domain, for example, in predicting sepsis in hospital patients [53], [54], and in jointly modeling patients' vital signals through multi-output GPs [55]. In future research, we will to extend our approach to multi-output GP models, and implement kernel functions customized for health care scenarios. By combining fast inference with flexible modeling, these approaches will have a profound impact in real-time monitoring and decision-making in patient health.

REFERENCES

[1] C. E. Rasmussen and C. K. I. Williams, *Gaussian Processes for Machine Learning*. Cambridge, MA, USA: MIT, 2006.

[2] M. M. Zhang and S. A. Williamson, "Embarrassingly parallel inference for Gaussian processes," *J. Mach. Learn. Res.*, vol. 20, no. 169, pp. 1–26, 2019.

[3] E. Snelson and Z. Ghahramani, "Sparse Gaussian processes using pseudo-inputs," in *Proc. Adv. Neural Inf. Process. Syst.*, 2005, pp. 1257–1264.

[4] M. K. Titsias, "Variational learning of inducing variables in sparse Gaussian processes," in *Proc. Int. Conf. Artif. Intell. Statist.*, 2009, pp. 567–574.

[5] J. Hensman, N. Fusi, and N. D. Lawrence, "Gaussian processes for Big Data," in *Proc. Uncertainty Artif. Intell.*, 2013, pp. 1–9.

[6] M. P. Deisenroth and J. W. Ng, "Distributed Gaussian processes," in *Proc. Int. Conf. Mach. Learn.*, 2015, pp. 1481–1490.

[7] S. Cohen, R. Mbuvha, T. Marwala, and M. Deisenroth, "Healing products of Gaussian process experts," in *Proc. Int. Conf. Mach. Learn.*, 2020, pp. 2068–2077.

[8] R. B. Gramacy and H. K. H. Lee, "Bayesian treed Gaussian process models with an application to computer modeling," *J. Amer. Stat. Assoc.*, vol. 103, no. 483, pp. 1119–1130, 2008.

[9] C. E. Rasmussen and Z. Ghahramani, "Infinite mixtures of Gaussian process experts," in *Proc. Adv. Neural Inf. Process. Syst.*, 2002, pp. 881–888.

[10] E. Meeds and S. Osindero, "An alternative infinite mixture of Gaussian process experts," in *Proc. Adv. Neural Inf. Process. Syst.*, 2006, pp. 883–890.

[11] V. Tresp, "Mixtures of Gaussian processes," *Adv. Neural Inf. Process. Syst.*, vol. 13, pp. 654–660, 2000.

[12] C. Yuan and C. Neubauer, "Variational mixture of Gaussian process experts," in *Proc. Adv. Neural Inf. Process. Syst.*, 2009, pp. 1897–1904.

[13] S. Sun and X. Xu, "Variational inference for infinite mixtures of Gaussian processes with applications to traffic flow prediction," *IEEE Trans. Intell. Transp. Syst.*, vol. 12, no. 2, pp. 466–475, Jun. 2011.

[14] T. Nguyen and E. Bonilla, "Fast allocation of Gaussian process experts," in *Proc. Int. Conf. Mach. Learn.,*2014, pp. 145–153.

[15] M. Trapp, R. Peharz, F. Pernkopf, and C. E. Rasmussen, "Deep structured mixtures of Gaussian processes," in *Proc. Int. Conf. Artif. Intell. Statist.,* 2020, pp. 2251–2261.
[16] D. Nguyen-Tuong, J. R. Peters, and M. Seeger, "Local Gaussian process regression for real time online model learning," in *Proc. Adv. Neural Inf. Process. Syst.*, 2009, pp. 1193–1200.
[17] L. Csató and M. Opper, "Sparse on-line Gaussian processes," *Neural Computation*, vol. 14, no. 3, pp. 641–668, 2002.
[18] T. D. Bui, C. Nguyen, and R. E. Turner, "Streaming sparse Gaussian process approximations," in *Proc. Adv. Neural Inf. Process. Syst.*, 2017, pp. 3299–3307.
[19] H. Robbins and S. Monro, "A stochastic approximation method," *Ann. Math. Statist.*, vol. 22, no. 13, pp. 400–407, 1951.
[20] M. D. Hoffman, D. M. Blei, C. Wang, and J. Paisley, "Stochastic variational inference," *J. Mach. Learn. Res.*, vol. 14, no. 1, pp. 1303–1347, 2013.
[21] M. Welling and Y. W. Teh, "Bayesian learning via stochastic gradient Langevin dynamics," in *Proc. Int. Conf. Mach. Learn.,*2011, pp. 681–688.
[22] S. Stanton, W. Maddox, I. Delbridge, and A. G. Wilson, "Kernel interpolation for scalable online Gaussian processes," in *Proc. Int. Conf. Artif. Intell. Statist.*, 2021, pp. 3133–3141.
[23] A. Wilson and H. Nickisch, "Kernel interpolation for scalable structured Gaussian processes (KISS-GP)," in *Proc. Int. Conf. Mach. Learn.*, 2015, pp. 1775–1784.
[24] M. Schürch, D. Azzimonti, A. Benavoli, and M. Zaffalon, "Recursive estimation for sparse Gaussian process regression," *Automatica*, vol. 120, 2020, Art. no. 109127.
[25] K. Kersting, C. Plagemann, P. Pfaff, and W. Burgard, "Most likely heteroscedastic Gaussian process regression," in *Proc. Int. Conf. Mach. Learn.,*2007, pp. 393–400.
[26] V. Tolvanen, P. Jylänki, and A. Vehtari, "Expectation propagation for nonstationary heteroscedastic Gaussian process regression," in *Proc. IEEE Int. Workshop Mach. Learn. Signal Process.*, 2014, pp. 1–6.
[27] M. Heinonen, H. Mannerström, J. Rousu, S. Kaski, and H. Lähdesmäki, "Non-stationary Gaussian process regression with Hamiltonian Monte Carlo," in *Proc. Int. Conf. Artif. Intell. Statist.*, 2016, pp. 732–740.
[28] C. J. Paciorek and M. J. Schervish, "Spatial modeling using a new class of nonstationary covariance functions," *Environmetrics*, vol. 17, no. 5, pp. 483–506, 2006.
[29] Y. Saatçi, R. D. Turner, and C. E. Rasmussen, "Gaussian process change point models," in *Proc. Int. Conf. Mach. Learn.,*2010, pp. 927–934.
[30] R. P. Adams and D. J. C. MacKay, "Bayesian online changepoint detection," 2007, *arXiv:0710.3742*.
[31] T. S. Ferguson, "A Bayesian analysis of some nonparametric problems," *Ann. Statist.*, vol. 1, no. 2, pp. 209–230, 1973.
[32] D. J. Aldous, "Exchangeability and related topics," in *École d'été de Probabilités de Saint-Flour XIII-1983*. Berlin, Germany: Springer, 1985, pp. 1–198.
[33] J. Pitman, "Combinatorial stochastic processes," Lecture Notes for St Flour Course, Dept. Statist., UC Berkeley, Berkeley, CA, USA, Tech. Rep. 621, 2002.
[34] P. Del Moral, A. Doucet, and A. Jasra, "Sequential Monte Carlo samplers," *J. Roy. Stat. Soc.: Ser. B. (Stat. Methodol.)*, vol. 68, no. 3, pp. 411–436, 2006.
[35] Y. Ulker, B. Günsel, and T. Cemgil, "Sequential Monte Carlo samplers for Dirichlet process mixtures," in *Proc. Artif. Intell. Statist.,*2010, pp. 876–883.
[36] C. M. Carvalho, H. F. Lopes, N. G. Polson, and M. A. Taddy, "Particle learning for general mixtures," *Bayesian Anal.*, vol. 5, no. 4, pp. 709–740, 2010.
[37] R. B. Gramacy and N. G. Polson, "Particle learning of Gaussian process models for sequential design and optimization," *J. Comput. Graphical Statist.*, vol. 20, no. 1, pp. 102–118, 2011.
[38] A. Svensson, J. Dahlin, and T. B. Schön, "Marginalizing Gaussian process hyperparameters using sequential Monte Carlo," in *Proc. Int. Workshop Comput. Adv. Multi-Sensor Adaptive Process.*, 2015, pp. 477–480.
[39] F. Hutter, H. H. Hoos, and K. Leyton-Brown, "Sequential model-based optimization for general algorithm configuration," in *Proc. Int. Conf. Learn. Intell. Optim.*, 2011, pp. 507–523.
[40] J. S. Bergstra, R. Bardenet, Y. Bengio, and B. Kégl, "Algorithms for hyper-parameter optimization," in *Proc. Adv. Neural Inf. Process. Syst.*, 2011, pp. 2546–2554.
[41] J. Snoek, H. Larochelle, and R. P. Adams, "Practical Bayesian optimization of machine learning algorithms," in *Proc. Adv. Neural Inf. Process. Syst.*, 2012, pp. 2951–2959.
[42] Z. Wang, M. Zoghi, F. Hutter, D. Matheson, and N. De Freitas, "Bayesian optimization in high dimensions via random embeddings," in *Proc. Int. Joint Conf. Artif. Intell.*, 2013, pp. 1778–1784.
[43] N. Srinivas, A. Krause, S. M. Kakade, and M. Seeger, "Gaussian process optimization in the bandit setting: No regret and experimental design," in *Proc. Int. Conf. Mach. Learn.*, 2010.
[44] L. Li, K. Jamieson, G. DeSalvo, A. Rostamizadeh, and A. Talwalkar, "Hyperband: A novel bandit-based approach to hyperparameter optimization," *J. Mach. Learn. Res.*, vol. 18, no. 1, pp. 6765–6816, 2017.
[45] D. Russo and B. Van Roy, "Learning to optimize via posterior sampling," *Math. Operations Res.*, vol. 39, no. 4, pp. 1221–1243, 2014.
[46] I. Murray, R. Adams, and D. MacKay, "Elliptical slice sampling," in *Proc. Int. Conf. Artif. Intell. Statist.,*2010, pp. 541–548.
[47] I. Murray and R. P. Adams, "Slice sampling covariance hyperparameters of latent Gaussian models," *Adv. Neural Inf. Process. Syst.*, vol. 23, pp. 1732–1740, 2010.
[48] R. M. Neal, "Markov chain sampling methods for Dirichlet process mixture models," *J. Comput. Graphical Statist.*, vol. 9, no. 2, pp. 249–265, 2000.
[49] M. D. Escobar and M. West, "Bayesian density estimation and inference using mixtures," *J. Amer. Stat. Assoc.*, vol. 90, no. 430, pp. 577–588, 1995.
[50] S. Minsker, S. Srivastava, L. Lin, and D. B. Dunson, "Scalable and robust Bayesian inference via the median posterior," in *Proc. Int. Conf. Mach. Learn.,*2014, pp. 1656–1664.
[51] S. Srivastava, V. Cevher, Q. Dinh, and D. B. Dunson, "WASP: Scalable Bayes via barycenters of subset posteriors," in *Proc. Int. Conf. Artif. Intell. Statist.*, 2015, pp. 912–920.
[52] A. E. Johnson et al., "MIMIC-III, a freely accessible critical care database," *Sci. Data*, vol. 3, 2016, Art. no. 160035.
[53] J. Futoma, S. Hariharan, and K. Heller, "Learning to detect sepsis with a multitask Gaussian process RNN classifier," in *Proc. Int. Conf. Mach. Learn.*, 2017, pp. 1174–1182.
[54] J. Futoma et al., "An improved multi-output Gaussian process RNN with real-time validation for early sepsis detection," in *Proc. Mach. Learn. Healthcare Conf.*, 2017, pp. 243–254.
[55] L.-F. Cheng et al., "Sparse multi-output Gaussian processes for online medical time series prediction," *BMC Med. Informat. Decis. Mak.*, vol. 20, no. 1, pp. 1–23, 2020.

Michael Minyi Zhang is currently an Assistant Professor with the Department of Statistics and Actuarial Science, The University of Hong Kong, Hong Kong. His research interests include statistical machine learning, scalable inference, and Bayesian non-parametrics.

Bianca Dumitrascu is currently an Assistant Professor with the Department of Statistics and the Herbert and Florence Institute for Cancer Dynamics, Columbia University, New York, NY, USA. Her research interests include developing statistical and computational methods to characterize the interplay between morphology and genomics in the early development of organisms.

Sinead A. Williamson is currently an Associate Professor with the Department of Statistics and Data Science, The University of Texas at Austin, Austin, TX, USA. Her research interests include network analysis, scalable inference methods, and Bayesian non-parametrics.

Barbara E. Engelhardt is currently a Professor with the Department of Biomedical Data Science, Stanford University, Stanford, CA, USA. Her research interests include developing statistical models and methods for the analysis of high-dimensional biomedical data, with a goal of understanding the underlying biological mechanisms of complex phenotypes and human disease.